High-Quality Software Engineering

Part 1: Quality Is Our Life (Step 1)

Write Down Your Answers

1. What is software engineering?
2. What is high-quality software?
3. What do we need to build it?

No wrong answers yet. We compare at the end of Part 3.

The Story in One Line

Software engineering is **managing complexity**
to build **high-quality software**
with **rules** and **tools**.

The 5 Steps

1. High-quality SE is our **life**
2. The essence: **managing complexity**
3. Our answer: **rules** and **tools**
4. **Tools**: what we use
5. **Rules**: how we decide

The 3 Parts

- **Part 1**: Step 1 (this part)
- **Part 2**: Steps 2-3
- **Part 3**: Steps 4-5

Step 1

High-quality software engineering is our life.

Not just "software".

Homework vs. Product

	Homework	Real product
Users	You	Millions
Lifetime	1 week	Years
If it breaks	Lose points	Lose money
Builder	You	A team

We Build Products

- A product is software people **pay** for.
- It must work, on time, within budget.
- It must **keep working** for years.

Have You Ever...

- Deleted an app that **crashed**?
- Left an app that was **too slow**?
- Quit an app that was **hard to use**?

Users **leave** when a better app appears.

Quality is **our job**.

What Is Quality?

- **Functionality**: does what users need
- **Reliability**: keeps working
- **Performance**: fast enough
- **Usability**: easy to use
- **Security**: safe
- **Maintainability**: safe to change

We Must Choose

Quality means different things to different people.

- All six attributes matter.
- We cannot cover all of them.
- So we **choose one**: maintainability.

Our Focus: Maintainability

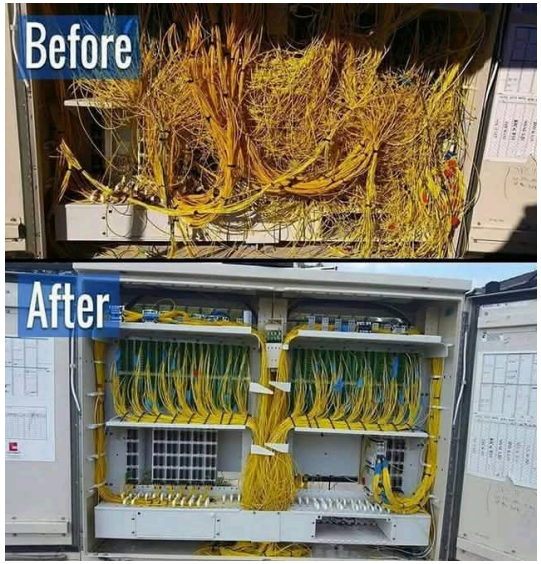
Software is never finished. We always:

1. **Fix bugs**
2. **Add features**

Why Maintainable?

To fix bugs and add features safely, we need **maintainable** software.

So we use maintainability as our measure of **high quality**.



Why Maintainability First?

When it breaks down, everything else follows.

- Hard to change → new bugs
- New bugs → less reliable
- Less reliable → users leave

High quality = **safe to change**.

Hard to Change

```
def price_student(p): return p * 0.9 * 1.07
def price_staff(p):   return p * 0.8 * 1.07
def price_guest(p):   return p * 1.07
```

Tax changes to 8%. Fix **3 places**. Miss one: **new bug**.

Easy to Change

```
TAX = 1.07
DISCOUNT = {"student": 0.9, "staff": 0.8, "guest": 1.0}

def price(p, role):
    return p * DISCOUNT[role] * TAX
```

- Tax change: **1 line**
- New role: **1 entry**

Same Answer, Different Quality

Both versions give the **same price** today.

Only one is **high quality**.

Quality shows up when we **change** the software.

Step 1 Summary

- We build **products**.
- Quality = works **and** easy to change.

But Why Is It Hard?

Keeping software easy to change is **not easy**.

Something fights against us.

*Part 2: **What is the enemy?***